

SILAB[®]

Änderungen/Neuerungen in SILAB[®] 2.5

Inhalt

1	INKOMPATIBLE ÄNDERUNGEN	3
2	SILAB FRAMEWORK	4
2.1	Erweiterte Multithreading-Unterstützung	4
2.2	Benutzeroberfläche	4
2.3	Erweiterung von SILAB	5
2.3.1	Neues DPU-Modell	5
2.3.2	Erweiterung von SILAB mit Ruby-Skripten	5
3	DATENBASIS SCNX	5
3.1	Schildergenerierung	5
3.1.1	Schilder einfügen über den Objects-Abschnitt von Course-Streckentypen	5
3.1.2	TrafficRules-Abschnitt von Course-Streckentypen	7
3.1.3	Schildergenerierung in Area2-Streckentypen	8
4	VERKEHRSSIMULATION TRFX	9
5	ANDERE DPUS	10
5.1	DPUMMI	10

1 INKOMPATIBLE ÄNDERUNGEN

- Die **Darstellung von Häusern** wurde aktualisiert, so dass diese bei Nacht beleuchtet werden können. Um Darstellungsfehler zu verhindern, müssen daher **alle Area2-Szenarien** mit der aktuellen Version von SILABAEEdit **erneut exportiert** werden.
- Zur Erhöhung der Kompatibilität und zur Vorbereitung der Portierung von SILAB auf 64-Bit-Systeme wurde das **Netzwerkprotokoll** für die **Fernsteuerung von Rechnern mit SILABAdmin** und SILABButler (nicht aber das Protokoll für den Filetransfer in SILABAdmin) **geändert**. Soll das Update auf SILAB Version 2.5 mit SILABAdmin eingespielt werden, muss daher folgende Vorgehensweise beachtet werden:
 - Zunächst vom Operatorrechner aus mit SILABAdmin alle Simulatorrechner hochfahren, und warten, bis diese komplett gestartet sind. SILABAdmin und alle anderen SILAB-Programme nun auf dem Operatorrechner beenden.
 - Erst jetzt auf dem Operatorrechner zunächst die neue SILAB-Version einspielen (z.B. mit dem Installer)
 - SILABAdmin auf dem Operator starten. Der Status aller Remote-Rechner wird wegen der inkompatiblen Version als ‚?‘ angezeigt.
 - System-Update des „bin“-Verzeichnisses ohne Unterverzeichnisse auf alle Simulatorrechner durchführen.
 - Alle Simulatorrechner manuell (z.B. mit VNC oder Power-Schalter) neustarten.
 - Nach dem Neustart sollte der Status der Remoterechner in SILABAdmin korrekt angezeigt werden. Das Update des Restsystems (lib- und Projects-Verzeichnisse) kann nun durchgeführt werden.
- Die Ausgangsvariablen der **DPUmstVeda** bzw. der **DPUVehicleDynamics** wurden angepasst.
 - Die neuen Ausgangs-Variablen
vx, vy, vz
enthalten die Geschwindigkeit des Fahrzeugs relativ zu seiner Längs-, Quer- bzw. Drehachse.
 - Die bisher vorhandenen Ausgangsvariablen
vX, vY und vZ
wurden umbenannt in
v_X, v_Y und v_Z.
Diese Variablen enthalten die Geschwindigkeit des Fahrzeugs relativ zur X-, Y- und Z-Achse des ortsfesten ("Welt-")Koordinatensystems und dürften nur selten von Interesse sein.

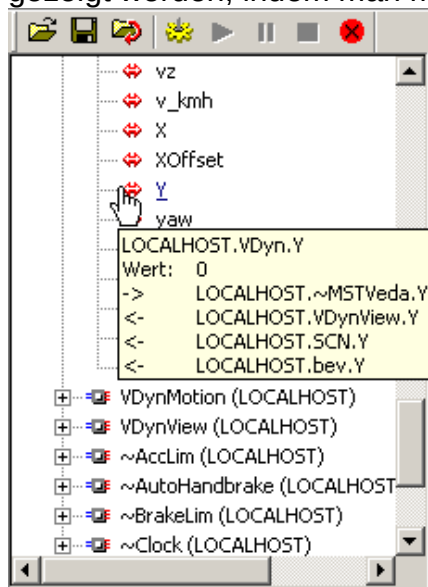
2 SILAB FRAMEWORK

2.1 Erweiterte Multithreading-Unterstützung

- Einzelne DPUs können jetzt parallel in unterschiedlichen Threads ausgeführt werden, um Systeme mit mehreren Prozessorkernen besser auszulasten.
- In der SILAB-Version 2.5 wird allerdings Multithreading noch nicht für alle DPUs unterstützt. Alle DPUs, die nur über DPU-Variablen miteinander kommunizieren, wie z.B. generische DPUs (DPUAmplifier) können frei auf mehrere Threads verteilt werden. Dagegen müssen die Simulations-Kernkomponenten (DPUSCNX, DPUTRFX, DPUMOV, usw.) alle im selben Thread ausgeführt werden.
- Mit dem DPU-Entwicklerpaket erstellte DPUs müssen in der aktuellen Version 2.5 im selben Thread wie die Kernkomponenten (DPUSCNX, DPUTRFX, usw.) ausgeführt werden, wenn eine der Schnittstellen iSCNX, iTRF oder ODB verwendet wird. Findet hingegen die Kommunikation mit anderen DPUs nur über DPU-Variablen statt, können sie frei auf zusätzliche Threads verteilt werden.
- Näheres hierzu finden Sie im Dokument „SILAB Multithreading“.

2.2 Benutzeroberfläche

- Die Verknüpfungen von Variablen können im Explorerfenster von SILAB angezeigt werden, indem man mit der Maus auf eine Variable zeigt.



- Wenn die ausgewählte Variable (hier LOCALHOST.VDyn.Y) Ziel einer Verknüpfung ist, wird die Quelle der Verknüpfung angezeigt als
→ (Quellvariable)
Wenn die ausgewählte Variable Quelle einer oder mehrerer Verknüpfungen ist, werden die Ziele dieser Verknüpfungen aufgelistet als
← (Zielvariable)
- Für lokale Variablen sowie für alle Variablen, für die ein Display geöffnet ist, wird außerdem der aktuelle Wert der Variablen angezeigt.

2.3 Erweiterung von SILAB

2.3.1 Neues DPU-Modell

Für SILAB-Erweiterungs-Module, die in der Programmiersprache C++ erstellt werden („DPU“), wird ein neues, vereinfachtes Programmiermodell eingeführt. Das neue Modell („SPU“, SILAB Program Unit) wird im Dokument „SILAB Erweiterung um DPUs“ ausführlich erläutert.

Vorhandene DPUs, die nach dem alten Programmiermodell erstellt wurden, sind unverändert weiterhin lauffähig.

2.3.2 Erweiterung von SILAB mit Ruby-Skripten

Seit der Version SILAB 2.1 können auch Skripte, die in der Programmiersprache Ruby erstellt wurden, in die Simulation eingebunden werden.

Die vollständig objektorientierte Programmiersprache Ruby ist leicht zu erlernen und bietet deutlich mehr Möglichkeiten als das früher verwendete Silascript.

Näheres zur Einbindung von Ruby-Skripten können Sie in den Dokumenten „DPURubyScript“ und „SILAB Ruby API“ nachlesen.

In der SILAB-Version 2.5 wurde die Ruby-Schnittstelle in SILAB nochmals deutlich erweitert: Die Eingangs- und Ausgangsvariablen eines Skriptes können Defaultwerte erhalten, automatisch verknüpft werden, und es sind auch Zeichenketten zulässig, außerdem können Informationen aus der Konfigurationsdatei bzw. zur aktuell laufenden Simulation erhalten werden (Modul „SILAB“). Die neuen Funktionen sind in „SILAB Ruby API“, Kap. 3 und 4 dokumentiert.

3 DATENBASIS SCNX

3.1 Schildergenerierung

- Die grafische Beschreibung der Verkehrsschilder wird nun automatisch beim Start der Simulation generiert. Dadurch können neue Schilder einfach hinzugefügt werden, indem die Textur des Schilds im Unterverzeichnis `sign` des Projektverzeichnisses (oder auch des SILAB-Texturverzeichnisses) abgelegt wird. Größe und Aufbau der Schilder (z.B. Position und Anzahl von Schildpfosten) sind im Abschnitt `SCNX SignGeneration` der Datei `\SILAB\lib\configurations\SCNX\SCNX.cfg` definiert. Falls das Schild ein Sonderformat hat, muss es evtl. in die zugehörige Gruppe aufgenommen werden.

3.1.1 Schilder einfügen über den Objects-Abschnitt von Course-Streckentypen

- Zur Platzierung eines Schilds an einem Course-Streckentypen wird dieses im Objects-Abschnitt wie folgt eingefügt:

```
Objects = {
    (sign.<Size_><Pos_><Number>, <Streckenmeter>, <Querposition>, <Drehwinkel>)
    ...
};
```
- Die eigentliche Schilddefinition wird dabei als `<Size_><Pos_>Number` angegeben. Dies bedeutet im einzelnen:

- Die Size-Angabe bezeichnet die Größe des Schilds. Sie kann entweder in cm angegeben werden („Sign100“ bedeutet Schildbreite ist 100 cm) oder es können die vordefinierten Größen „big“, „middle“ und „small“ verwendet werden. Die Size-Angabe kann auch weggelassen werden. In diesem Fall wird die Größe des Schilds automatisch anhand der Entwurfsgeschwindigkeit, welche im zugehörigen CourseType der Strecke festgelegt wurde, bestimmt.
- Die Pos-Angabe gibt die Position des Schilds an. Dabei bestehen folgende Möglichkeiten:
 - (keine Angabe): Schild wird in normaler Höhe am Pfosten aufgehängt.
 - N: Schild wird in niedriger Höhe am Pfosten aufgehängt. Diese Position ist für „An dieser Verkehrsinsel vorbeifahren“-Schilder geeignet.
 - H: Schild wird oben am Pfosten aufgehängt. Es wird sichergestellt, dass ein Zusatzschild unterhalb Platz findet.
 - U: Schild wird in niedriger Höhe platziert, es wird kein Pfosten aufgestellt. Diese Position sollte für Zusatzschilder verwendet werden.
- Die Number-Angabe gibt die Nummer des Schilds nach StVO, evtl. zusätzlich mit einer Varianten- oder Beschriftungsangabe an. Sie entspricht dem Dateinamen (ohne Dateierweiterung) der Texturdatei.
- Einige Beispiele für Schilddefinitionen:

```
Objects = {  
    (sign.205, 20, 5),           # (1) Vorfahrt achten  
    (sign.H_278_060, 40, 5),    # (2a) Geschwindigkeitsbeschränkung  
    (sign.U_1002_24a, 40, 5),   # (2b) Zusatzschild  
    (sign.Sign300_N_222_020, 60, 3) # (3) Vorbeifahren  
};
```



- Bei Schild 1 [(sign.205, 20, 5)] wird Standardgröße und –Position verwendet. ‚Vorfahrt achten‘ ist Schild Nr. 205. Das Schild wird an Streckenmeter 20 platziert und 5 m rechts neben die Mittellinie der Straße gesetzt.
- Schild 2a [(sign.H_278_060, 40, 5)] und 2b [(sign.U_1002_24a, 40, 5)] hängen am selben Pfosten. Dies wird erreicht, indem für beide Schilder dieselbe Position (Streckenmeter 40; 5 m nach rechts) verwendet wird und das erste Schild („60 aufgehoben“) in der Position „H“ bzw. das zweite Schild („abknickende Vorfahrtsstraße“) als Zusatzschild, Position „U“ platziert wird.
- Bei Schild 3 [(sign.Sign300_N_222_020, 60, 3)] wird die (sehr große) Schildbreite 300 cm festgelegt und das Schild wird auf niedriger Position befestigt.

3.1.2 TrafficRules-Abschnitt von Course-Streckentypen

Schilder können auch über den neuen TrafficRules-Abschnitt von Course-Streckentypen eingefügt werden.

Hierdurch ist es möglich, zusätzliche Informationen über das Schild anzugeben, so dass der Fremdverkehr korrekt auf das Schild reagieren kann.

Ein Beispiel:

```
TrafficRules T
{
  Sign S { Sign = ("330", "274_100"); s = 20; d = 5; };
  Sign X { Sign = ("101"); s = 40; d = 5; Dir = TOWARDS; };
```

} ;

Hierdurch werden zwei Schilder platziert, wie im Screenshot ersichtlich.



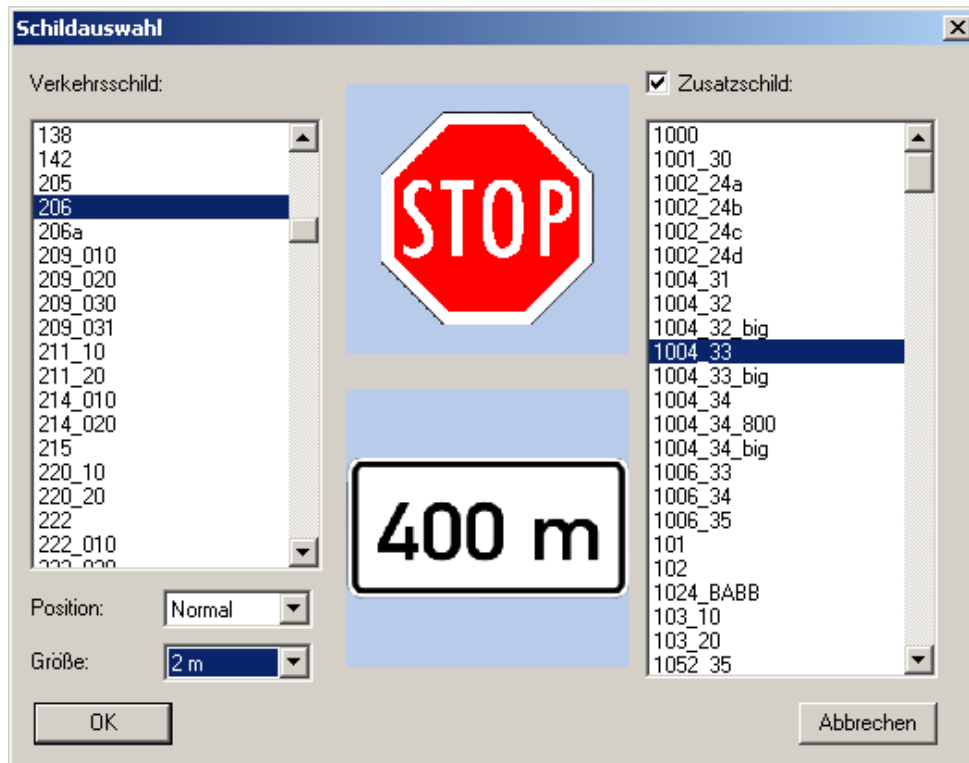
Die Schildtypen werden von oben nach unten in der Liste „Sign“ aufgezählt. (Bis zu drei Schilder können an einem Pfosten platziert werden.) Position und seitlicher Abstand zur Straßenmittellinie werden über die „s“ und „d“ Parameter festgelegt. Durch die Angabe von Dir = TOWARDS kann das Schild für die Gegenrichtung definiert werden.

Die genaue Syntaxdefinition des TrafficRules-Abschnitts findet sich in der „SILAB Referenz zu SCNX“.

3.1.3 Schildergenerierung in Area2-Streckentypen

Zur Platzierung von Schildern in Area2-Streckentypen kann das neue Werkzeug

„Schild einfügen“  im Layer „Grafik 2“ in SILABAEEdit verwendet werden. Im erscheinenden Dialog kann ein Schild mit optionalem Zusatzschild zusammengestellt und eingefügt werden.



Durch einen Doppelklick auf das eingefügte Schild können dessen Eigenschaften nachträglich geändert werden.

4 VERKEHRSSIMULATION TRFX

- Es sind neun neue Fahrzeugmodelle hinzugekommen. Dies sind:
 - cars.Alfa147GTA
 - cars.BMWm3
 - cars.Camaro
 - cars.ClioV6
 - cars.Corolla
 - cars.HondaCRX
 - cars.MiniCooper
 - cars.Octavia
 - cars.ToyotaSprinter
- Die neuen Fahrzeugmodelle sind jeweils in mehreren Farbvarianten verfügbar. Eine Referenz der Eigenschaften und Screenshots dieser Modelle finden Sie in der „SILAB Referenz TRFX“.
- Das neue Verhaltensmodell ‚EXIDM‘ ermöglicht die Simulation von Fremdfahrzeugen, die auf Autobahnen automatisch eine geeignete Spur wählen, und Überholmanöver durchführen. Das Verhaltensmodell ist in der „SILAB Referenz TRFX“ beschrieben.

5 ANDERE DPUS

5.1 DPUMMI

- Die DPUMMI kann jetzt auf demselben Rechner mehrfach gestartet werden. Es können also mehrere Instanzen der DPUMMI in der DPU-Konfiguration eingebunden werden und demselben Rechner zugewiesen werden.
- Außerdem stehen die neuen Anzeige-Elemente „Scope“ (Oszilloskop) und „Group“ (logische Gruppierung) zur Verfügung. Diese sind in der Dokumentation „DPUMMI“ ausführlich beschrieben.